

Roadmap Completo — GridLoad Intelligence BR

Plano de desenvolvimento do zero até uma versão validada, documentada e apresentável

Princípio central: não avançar para Machine Learning antes de validar bem os dados e construir baselines confiáveis.

Visão geral

```
FASE 0 – Definição
↓
FASE 1 – Estrutura do projeto
↓
FASE 2 – Coleta dos dados
↓
FASE 3 – Limpeza e integração
↓
FASE 4 – EDA
↓
FASE 5 – Feature Engineering
↓
FASE 6 – Baselines + validação temporal
↓
FASE 7 – Load Forecast
↓
FASE 8 – Peak Intelligence
↓
FASE 9 – Ramp Intelligence
↓
FASE 10 – Anomaly Detection
↓
FASE 11 – Weather Intelligence
↓
FASE 12 – Explainable AI
↓
FASE 13 – GridAlert
↓
FASE 14 – Dashboard
↓
FASE 15 – Validação final
↓
FASE 16 – Documentação e apresentação
```

FASE 0 — Congelar o escopo

Objetivo

Impedir que o projeto cresça indefinidamente.

Antes de programar, defina oficialmente:

Projeto

GridLoad Intelligence BR

Problema principal

Prever e interpretar o comportamento futuro da carga elétrica de um subsistema brasileiro.

Targets

Forecast:

```
t + 1h
t + 6h
t + 24h
```

Classificação:

pico nas próximas 3h
rampa elevada nas próximas 3h

Anomalia:

carga real muito diferente
da carga esperada

Produto final:

GridAlert

NORMAL
ATENÇÃO
CRÍTICO

Escolher apenas um subsistema inicialmente

Eu começaria com: Nordeste

Motivo: reduz enormemente a complexidade inicial.

Depois você pode adicionar:

Norte
Sul
Sudeste/Centro-Oeste

como extensão.

O que NÃO entra

Deixe registrado no README desde o início:

- × Grid Balance Intelligence
- × previsão de geração solar
- × previsão de geração eólica
- × clustering
- × change point detection
- × otimização do SIN
- × despacho
- × baterias
- × demand response
- × deep learning obrigatório

Fase 0 concluída quando

Você conseguir explicar o projeto em aproximadamente 30 segundos:

O GridLoad Intelligence BR usa dados reais do sistema elétrico brasileiro para prever carga em diferentes horizontes, antecipar picos e rampas, identificar comportamentos atípicos e transformar essas informações em alertas interpretáveis pelo módulo GridAlert.

FASE 1 — Criar a estrutura do projeto

1. Criar repositório

gridload-intelligence-br/

Estrutura:

```

gridload-intelligence-br/
├── data/
│   ├── raw/
│   ├── interim/
│   └── processed/
├── notebooks/
│   ├── 01_data_inventory.ipynb
│   ├── 02_data_cleaning.ipynb
│   ├── 03_eda.ipynb
│   ├── 04_feature_engineering.ipynb
│   ├── 05_baseline.ipynb
│   ├── 06_forecasting.ipynb
│   ├── 07_peak_intelligence.ipynb
│   ├── 08_ramp_intelligence.ipynb
│   ├── 09_anomaly_detection.ipynb
│   ├── 10_weather_intelligence.ipynb
│   └── 11_explainability.ipynb
├── src/
│   ├── data/
│   ├── features/
│   ├── models/
│   └── alerts/
├── models/
├── dashboard/
├── reports/
├── tests/
├── requirements.txt
└── README.md

```

2. Criar ambiente Python

Bibliotecas iniciais:

```

pandas
numpy
matplotlib
plotly
scikit-learn
xgboost
lightgbm
shap
jupyter

```

Depois:

```

streamlit

```

Não instale tudo de uma vez sem necessidade.

FASE 2 — Inventário e coleta dos dados

Essa fase é extremamente importante.

Crie:

```

01_data_inventory.ipynb

```

2.1 ONS — carga elétrica

Baixe inicialmente alguns anos.

Sugestão:

```

2022
2023
2024
2025
2026

```

Não precisa começar em 2000.

Cinco anos de dados horários já representam aproximadamente:

$$5 \times 365 \times 24 \approx 43.800$$

registros por subsistema. Mais que suficiente para começar.

Verificar

- nome das colunas;
- resolução temporal;
- timezone;
- subsistema;
- unidade da carga;
- valores ausentes;
- duplicados;
- mudanças de formato entre anos.

2.2 INMET

Não tente integrar o Brasil inteiro imediatamente.

Escolha algumas estações representativas do Nordeste.

Comece principalmente com:

```
temperatura
umidade
```

Precipitação pode entrar depois.

2.3 Calendário

Crie posteriormente:

```
hora
dia da semana
mês
fim de semana
feriado
```

Para feriados, utilize calendário brasileiro.

Produto da Fase 2

Um documento/tabela como:

Base	Período	Frequência	Variáveis	Problemas
ONS carga	2022-2026	horária	carga	verificar missing
INMET	2022-2026	horária	temperatura, umidade	estações
calendário	derivado	horária	data/hora	—

Só avance quando souber exatamente o que possui.

FASE 3 — Limpeza dos dados

Notebook:

```
02_data_cleaning.ipynb
```

3.1 Padronizar timestamp

Objetivo:

```
2025-05-10 18:00:00
```

como índice temporal padronizado.

Verifique:

```
df["timestamp"].dtype
```

e ordene:

```
timestamp crescente
```

3.2 Filtrar subsistema

Inicialmente:

```
Nordeste
```

3.3 Verificar duplicados

Perguntas:

```
Existe mais de uma observação  
para o mesmo timestamp?
```

3.4 Missing values

Calcular:

```
% missing por coluna
```

Analise antes de simplesmente preencher.

Exemplo:

```
carga:      0.1%  
temperatura: 4.7%  
umidade:    5.2%
```

3.5 Outliers evidentes

Não elimine automaticamente valores altos de carga.

Eles podem justamente ser:

```
picos reais
```

Remova apenas valores claramente inválidos.

Produto da fase

Salvar:

```
data/interim/load_clean.parquet  
data/interim/weather_clean.parquet
```

Prefira Parquet para dados processados.

FASE 4 — Integrar ONS + INMET

Agora você começa a criar o dataset principal.

Granularidade:

```
1 linha = 1 hora
```

Dataset inicial

```
timestamp  
load_mw  
temperature  
humidity
```

Depois:

```
hour  
weekday  
month  
is_weekend  
is_holiday
```

Uma dificuldade importante

Você provavelmente terá várias estações meteorológicas.

Não complique inicialmente.

Pode criar:

```
| temperatura média regional
```

das estações escolhidas. Posteriormente pode melhorar.

Produto

```
| data/processed/gridload_base.parquet
```

FASE 5 — EDA

Notebook:

```
| 03_eda.ipynb
```

Essa fase precisa ser profunda.

5.1 Entender a variável principal

Calcule:

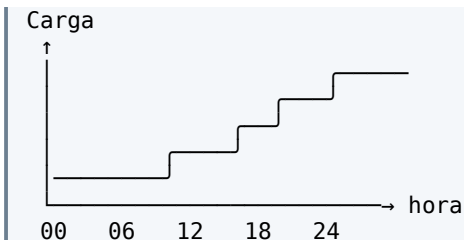
```
| min  
| max  
| mean  
| median  
| std  
| percentis
```

5.2 Curva de carga média diária

Agrupe por:

```
| hour
```

Você provavelmente verá algo semelhante:



5.3 Dia da semana

Compare:

```
| segunda  
| terça  
| quarta  
| quinta  
| sexta  
| sábado  
| domingo
```

5.4 Dia útil × fim de semana

Muito importante.

5.5 Sazonalidade mensal

Compare:

Jan
Fev
Mar
...
Dez

5.6 Histórico anual

Analise:

2022
2023
2024
2025
2026

Não para fazer change point detection. Apenas para entender o dado.

5.7 Temperatura × carga

Faça gráficos:

temperature
×
load

Também por:

hora
mês

5.8 Autocorrelação

Verifique relação com:

t-1h
t-2h
t-24h
t-48h
t-168h

É provável que:

t-24h
t-168h

tenham muito valor preditivo.

5.9 Rampas

Crie:

$$Ramp_t = L_t - L_t(t-1)$$

Analise:

média
desvio
P90
P95
P97
P99

5.10 Picos

Analise os percentis de carga.

Exemplo:

P90
P95
P97
P99

Isso ajudará a definir o Peak Intelligence.

Entregável da EDA

Ao final, produza um arquivo:

```
| reports/eda_summary.md
```

com aproximadamente:

```
| 10–15 descobertas principais
```

Exemplo:

```
| A carga apresenta padrão diário pronunciado, com maior concentração de valores elevados entre X e Y horas.
```

Não basta gerar gráficos. Você precisa escrever conclusões.

FASE 6 — Feature Engineering

Notebook:

```
| 04_feature_engineering.ipynb
```

6.1 Features calendário

```
hour
weekday
month
is_weekend
is_holiday
```

6.2 Encoding cíclico

Hora:

$$\text{hour_sin} = \sin(2\pi h/24)$$
$$\text{hour_cos} = \cos(2\pi h/24)$$

Também pode fazer para:

```
weekday
month
```

se achar necessário.

6.3 Lags

Comece com:

```
load_lag_1h
load_lag_2h
load_lag_3h

load_lag_24h
load_lag_48h

load_lag_168h
```

6.4 Rolling features

```
load_mean_3h
load_mean_6h
load_mean_24h

load_std_3h
load_std_6h
load_std_24h
```

6.5 Ramp

```
| ramp_1h
```

Opcional:

```
ramp_2h  
ramp_3h
```

6.6 Meteorologia

```
temperature  
humidity
```

Possivelmente:

```
temperature_lag_1h  
temperature_mean_6h
```

Não exagere.

6.7 Targets

Forecast:

```
target_load_1h  
target_load_6h  
target_load_24h
```

Criados com shift negativo.

Muito cuidado com data leakage.

FASE 7 — Estratégia de validação

Essa etapa vem antes de treinar qualquer modelo.

Nunca faça

```
train_test_split(shuffle=True)
```

Separação recomendada

Por exemplo:

```
2022-2024  
TRAIN  
  
2025  
VALIDATION  
  
2026  
TEST
```

Ou ajustar conforme os dados disponíveis.

Teste final intocável

Depois de definir:

```
TEST = 2026
```

não fique ajustando features olhando constantemente o resultado de 2026.

Use validation para isso.

FASE 8 — Baselines

Notebook:

```
05_baseline.ipynb
```

Essa etapa é obrigatória.

Baseline 1

Persistence:

$$\hat{L}_{(t+1)} = L_t$$

Baseline 2

Mesmo horário ontem:

$$\hat{L}_t = L_{(t-24)}$$

Baseline 3

Mesmo horário semana anterior:

$$\hat{L}_t = L_{(t-168)}$$

Métricas

Calcule:

```
MAE
RMSE
MAPE
```

Exemplo:

Modelo	MAE	RMSE	MAPE
t-1h	...	...	...
t-24h	...	...	...
t-168h	...	...	...

Esses valores serão sua referência.

FASE 9 — Load Forecast

Notebook:

```
06_forecasting.ipynb
```

Agora entra ML.

9.1 Linear Regression

Primeiro modelo real.

Isso verifica se:

```
features simples
```

já explicam bastante coisa.

9.2 Random Forest

Opcional, mas útil como comparação.

9.3 XGBoost ou LightGBM

Esse provavelmente será seu modelo principal.

Não precisa obrigatoriamente usar os dois.

Treinar separadamente para:

```
+1h
+6h
+24h
```

Você pode ter:

```
model_1h
model_6h
model_24h
```

Avaliação

Modelo	Horizonte	MAE	RMSE	MAPE
Baseline	1h			
XGBoost	1h			
Baseline	6h			
XGBoost	6h			
Baseline	24h			
XGBoost	24h			

Visualizações obrigatórias

Real × previsto

```
real —————  
previsto —————
```

Erro ao longo do tempo

Distribuição dos resíduos

Erro por hora

Erro por dia da semana

FASE 10 — Peak Intelligence

Notebook:

```
| 07_peak_intelligence.ipynb
```

Agora você utiliza o conhecimento da EDA.

10.1 Definir pico

Exemplo inicial:

```
| Peak = carga > P95
```

Mas teste:

```
| P90  
| P95  
| P97
```

Não escolha arbitrariamente.

10.2 Criar target

Pergunta: Existe um pico nas próximas 3 horas?

Então:

```
| peak_next_3h = 1
```

caso:

```
t+1  
ou  
t+2  
ou  
t+3
```

ultrapasse o limite.

10.3 Features

Use:

```
carga atual  
lags  
rolling  
ramp  
temperatura  
hora  
dia
```

10.4 Baseline

Uma regra simples:

```
se carga atual > P90  
→ peak risk
```

10.5 Modelo

Comece com:

```
| Logistic Regression
```

Depois:

```
| XGBoost classifier
```

10.6 Métricas

Não priorize accuracy.

Use:

```
Precision  
Recall  
F1  
PR-AUC
```

Também matriz de confusão.

FASE 11 — Ramp Intelligence

Notebook:

```
| 08_ramp_intelligence.ipynb
```

11.1 Definir rampa elevada

Baseado nos percentis.

Por exemplo:

```
| abs(ramp) > P95
```

ou somente:

```
| ramp positiva > P95
```

Dependendo da análise.

11.2 Target

```
| high_ramp_next_3h
```

11.3 Modelo

Mesma estratégia:

Logistic Regression
↓
XGBoost

11.4 Resultado esperado

Ramp Risk

Próximas 3h:

74%

e:

maior rampa prevista:
+820 MW/h

FASE 12 — Anomaly Detection

Notebook:

09_anomaly_detection.ipynb

Mantenha simples.

Método principal

Use os resíduos:

$$Residual = L_{real} - L_{previsto}$$

Depois normalize.

Exemplo:

$$Z = (Residual - \mu) / \sigma$$

Você pode considerar:

$|Z| < 2$
normal

$2 \leq |Z| < 3$
incomum

$|Z| \geq 3$
anormal

Os limites devem ser avaliados empiricamente.

Anomaly Score

Transforme isso em algo entre:

0 e 1

para o dashboard.

Extensão opcional

Somente depois:

Isolation Forest

Compare com o método baseado em resíduos.

FASE 13 — Weather Intelligence

Notebook:

10_weather_intelligence.ipynb

Essa fase é um experimento científico, não outro sistema.

Modelo A

```
histórico  
+  
calendário
```

Modelo B

```
histórico  
+  
calendário  
+  
temperatura  
+  
umidade
```

Compare

```
MAE  
RMSE  
MAPE
```

Pergunta: Meteorologia melhora significativamente a previsão?

Também analise

```
temperatura × carga
```

separando:

```
hora  
mês  
fim de semana
```

Produto

Uma conclusão objetiva.

Exemplo hipotético:

```
A inclusão de variáveis meteorológicas reduziu o MAE em 4,2% no horizonte de 24 horas, mas  
apresentou ganho marginal para previsões de uma hora.
```

Não procure forçar um resultado positivo.

FASE 14 — Explainable AI

Notebook:

```
11_explainability.ipynb
```

Agora utilize SHAP.

14.1 Explicabilidade global

Responder: Quais features mais influenciam o modelo?

Exemplo:

```
load_lag_1h  
load_lag_24h  
hour  
temperature  
load_lag_168h
```

14.2 Explicabilidade local

Escolha um momento específico.

Exemplo:

```
19:00  
forecast = 14.500 MW
```

Mostre quais features elevaram ou reduziram a previsão.

14.3 Peak model

Também aplique SHAP ao modelo de pico.

Pergunta: Por que o modelo considera que existe risco elevado de pico?

FASE 15 — Construir GridAlert

Agora chegamos ao produto final.

Essa etapa deve ser simples.

Inputs

```
forecast
peak_risk
ramp_risk
anomaly_score
```

Estados

```
NORMAL
ATENÇÃO
CRÍTICO
```

Primeira lógica

Exemplo:

```
NORMAL

peak < 50%
ramp < 50%
anomalia baixa

ATENÇÃO

peak elevado
OU
ramp elevada
OU
anomalia elevada

CRÍTICO

peak muito elevado
E
ramp muito elevada
```

Depois refine os thresholds utilizando validation.

O GridAlert também deve gerar explicação

Não apenas:

```
CRÍTICO
```

Mas:

GRIDALERT

STATUS
ATENÇÃO

PERÍODO
18h–20h

MOTIVOS

- Peak Risk = 84%
- Rampa prevista = P96
- Carga prevista acima do P95 histórico

ANOMALY
Normal

FASE 16 — Validar o GridAlert historicamente

Pegue seu período de teste.

Simule:

| hora por hora

como se estivesse no passado.

Para cada instante:

1. gerar forecast
2. calcular peak risk
3. calcular ramp risk
4. gerar GridAlert
5. esperar próximas horas
6. verificar o que realmente aconteceu

Métricas interessantes

Lead Time

| alerta 16h

| pico 19h

| lead time = 3h

False Alert Rate

| alertas emitidos
mas evento não aconteceu

Missed Events

Eventos importantes não alertados.

Essa validação é extremamente importante

Ela transforma:

| regra bonitinha

em:

| sistema avaliado historicamente

FASE 17 — Transformar notebooks em código

Só agora comece a tirar partes importantes dos notebooks.

Exemplo:

| src/features/build_features.py

```
src/models/forecast.py
src/models/peak.py
src/models/ramp.py
src/alerts/gridalert.py
```

Isso melhora muito a qualidade do GitHub.

FASE 18 — Salvar modelos

Exemplo:

```
models/
forecast_1h.joblib
forecast_6h.joblib
forecast_24h.joblib

peak_model.joblib
ramp_model.joblib
```

Também salve:

```
features utilizadas
versão do modelo
métricas
data de treinamento
```

FASE 19 — Dashboard

Agora sim.

Eu usaria: Streamlit + Plotly pela velocidade de desenvolvimento.

Página 1 — Overview

GRIDLOAD INTELLIGENCE BR

Subsistema:
Nordeste

Carga atual
12.840 MW

Forecast +1h
13.020 MW

Forecast +6h
14.310 MW

Forecast +24h
12.970 MW

GridAlert destacado

STATUS

ATENÇÃO

com:

Peak Risk	82%
Ramp Risk	71%
Anomaly	Normal

Página 2 — Forecast

Gráfico:

```
últimas 48h
+
próximas 24h
```

Com:

real
forecast

Página 3 — Analytics

- curva histórica;
- curva média diária;
- temperatura × carga;
- erros;
- rampas;
- picos;
- feature importance;
- SHAP.

Página 4 — Alerts

Horário	Status	Peak	Ramp	Motivo
14h	Normal	20%	14%	—
15h	Atenção	72%	51%	Peak
16h	Atenção	84%	69%	Peak
17h	Crítico	91%	86%	Peak + Ramp

FASE 20 — Replay histórico

Essa funcionalidade pode ficar muito boa.

Escolha um dia de teste.

O dashboard simula:

00h
↓
01h
↓
02h
...
↓
23h

como se estivesse recebendo dados em tempo real.

O usuário pode observar:

forecast mudando
peak risk mudando
ramp risk mudando
GridAlert mudando

Sem precisar construir infraestrutura real-time.

Isso já causa uma excelente impressão em demonstração.

FASE 21 — Testes

Teste funções importantes.

Por exemplo:

```
lags
rolling
target creation
peak threshold
ramp calculation
GridAlert rules
```

Especialmente para evitar: data leakage.

FASE 22 — Reprodutibilidade

Outra pessoa deve conseguir:

```
git clone
↓
pip install
↓
executar pipeline
↓
obter resultado
```

Tenha:

```
requirements.txt
README.md
config
```

e instruções claras.

FASE 23 — README

Eu estruturaria:

1. GridLoad Intelligence BR — descrição curta.
2. Problema — por que previsão/monitoramento de carga importa.
3. Arquitetura.
4. Dados — ONS e INMET.
5. EDA — principais descobertas.
6. Modelos — baseline e modelo final.
7. Resultados — métricas.
8. GridAlert — como funciona.
9. Dashboard — screenshots.
10. Limitações — importantíssimo.
11. Próximos passos.

Arquitetura resumida:

```
ONS + INMET
↓
Data Pipeline
↓
Features
↓
Forecast
↓
Peak/Ramp
↓
GridAlert
↓
Dashboard
```

FASE 24 — Relatório técnico

Além do GitHub, produza um PDF curto.

Algo como:

Estrutura:

Introdução
Problema
Dados
Metodologia
EDA
Feature Engineering
Forecast
Peak Intelligence
Ramp Intelligence
Anomaly Detection
Weather Intelligence
Explainability
GridAlert
Resultados
Limitações
Conclusões

FASE 25 — Limitações

Documente claramente.

Exemplos:

- projeto experimental;
- não representa ferramenta oficial do ONS;
- GridAlert não determina despacho;
- ausência de variáveis operacionais internas;
- previsão baseada em dados públicos;
- meteorologia regional simplificada;
- thresholds estatísticos experimentais.

Isso aumenta a credibilidade do projeto.

FASE 26 — Preparar apresentação

Prepare uma apresentação de aproximadamente:

8–12 slides

Narrativa:

1. Problema.
2. Dados brasileiros reais.
3. EDA.
4. Feature engineering.
5. Forecast.
6. Peak Intelligence.
7. Ramp Intelligence.

8. Explainability.
9. GridAlert.
10. Dashboard.
11. Resultados.
12. Conclusões.

Cronograma de 12 semanas

Semana	Principal objetivo	Entregável
1	Definição + coleta ONS	inventário
2	INMET + limpeza	dataset bruto validado
3	integração	dataset consolidado
4	EDA	relatório exploratório
5	feature engineering	dataset ML
6	baselines + forecast	modelo inicial
7	tuning + forecast final	modelos 1h/6h/24h
8	Peak Intelligence	classificador
9	Ramp + anomalies	módulos prontos
10	Weather + SHAP	interpretação
11	GridAlert + replay	sistema final
12	dashboard + README + relatório	projeto publicado

Crítérios de progresso

Checkpoint 1 — Dados

Você consegue responder:

Qual período?
 Qual frequência?
 Quantos missing?
 Qual subsistema?
 Quais estações meteorológicas?

Se não, não avance.

Checkpoint 2 — EDA

Você consegue explicar:

como a carga varia
 durante o dia
 durante a semana
 durante o ano

Se não, não treine ML.

Checkpoint 3 — Forecast

O ML supera:

```
t-1h  
t-24h  
t-168h
```

Se não superar, investigue antes de avançar.

Checkpoint 4 — Peak/Ramp

Precision/Recall fazem sentido?

Existem alertas úteis ou o modelo alerta tudo?

Checkpoint 5 — GridAlert

Ele produz:

```
status  
+  
motivo  
+  
período
```

e funciona em dados históricos não vistos.

Prioridades

Se o tempo começar a faltar:

Prioridade A — Obrigatório

```
Dados  
EDA  
Features  
Forecast  
Peak  
Ramp  
GridAlert  
Dashboard
```

Prioridade B — Importante

```
Weather Intelligence  
SHAP  
Replay histórico
```

Prioridade C — Só se sobrar tempo

```
Anomaly Detection avançado  
múltiplos subsistemas  
API  
PostgreSQL  
Docker  
automação de coleta
```

Ordem prática dos notebooks

Se quiser simplesmente seguir uma fila de execução:

```
01_data_inventory
↓
02_data_cleaning
↓
03_eda
↓
04_feature_engineering
↓
05_baseline
↓
06_forecasting
↓
07_peak_intelligence
↓
08_ramp_intelligence
↓
09_anomaly_detection
↓
10_weather_intelligence
↓
11_explainability
↓
GridAlert
↓
Dashboard
```

O principal cuidado técnico

Durante praticamente todo o projeto, faça esta pergunta:

Esta feature utiliza alguma informação que eu não teria disponível no instante em que estou fazendo a previsão?

Se sim, existe risco de data leakage.

Exemplo correto:

```
às 14h

usar carga:
13h
12h
ontem 14h
semana passada 14h
```

Exemplo incorreto:

```
às 14h

usar média calculada
com dados de 15h ou 16h
```

Esse provavelmente será o erro metodológico mais importante a evitar.

Resultado final esperado

Ao terminar os três meses, você deve conseguir abrir a aplicação e mostrar:

GRIDLOAD INTELLIGENCE BR

Subsistema
NORDESTE

Carga atual
12.840 MW

Previsão
+1h 13.020 MW
+6h 14.310 MW
+24h 12.970 MW

PEAK INTELLIGENCE
82% – ALTO

RAMP INTELLIGENCE
+920 MW/h – ELEVADA

ANOMALY
NORMAL

GRIDALERT
ATENÇÃO

Período relevante
18:00–20:00

Motivo
Pico e rampa elevados
previstos simultaneamente.

E, mais importante, você terá todo o caminho técnico por trás disso:

dados reais brasileiros → limpeza → EDA → engenharia de atributos → validação temporal → baseline
→ ML → classificação → explicabilidade → GridAlert → aplicação.

Esse é o roadmap que eu usaria como plano oficial de desenvolvimento do GridLoad Intelligence BR.